Aurora Siger - SIFSM

Sistema Integrado de Futuro Sustentável em Marte (SIFSM)

O Sistema Integrado de Futuro Sustentável em Marte (SIFSM) é um protótipo desenvolvido para gerenciar e analisar dados operacionais da colônia Aurora Siger, integrando informações de diferentes módulos e indicadores de consumo energético e condições ambientais. Desenvolvido em Python, o sistema utiliza banco de dados SQLite, operações CRUD, regras lógicas, estruturas de dados e simulações computacionais para monitorar recursos, identificar situações de risco e apoiar decisões operacionais. A solução também incorpora modelagem linear e uma proposta de arquitetura multiagente, demonstrando como a integração entre tecnologia, análise de dados e inteligência artificial pode contribuir para uma infraestrutura marciana mais eficiente, autônoma e sustentável.



Link do Repositório Público do GitHub: https://github.com/Miguelgonn/Aurora_Comunicacao

Integrantes:

Ana Gabriela rm571312

Kaique rm570533

Miguel Antunes rm573643

Miguel Gonçalves rm573793

Introdução

Descrição do projeto

A exploração e a ocupação de Marte representam desafios que exigem soluções tecnológicas capazes de garantir a sustentabilidade, a autonomia e a eficiência operacional de futuras colônias humanas. Em um ambiente com recursos limitados e condições extremas, o gerenciamento inteligente de energia, água, infraestrutura e condições ambientais torna-se fundamental para a manutenção da vida e para o desenvolvimento de atividades na superfície marciana.

Nesse contexto, a Aurora Siger propõe o desenvolvimento do Sistema Integrado de Futuro Sustentável em Marte (SIFSM), uma solução computacional voltada à organização, ao monitoramento e à análise de dados operacionais da colônia. O sistema busca integrar informações de diferentes módulos, identificar possíveis situações de risco, simular cenários de consumo de recursos e oferecer suporte às decisões relacionadas à sustentabilidade e à operação da infraestrutura.

Desenvolvido em Python, o protótipo reúne recursos de programação, banco de dados, estruturas de dados, lógica computacional, modelagem matemática e inteligência artificial. Por meio da integração desses conceitos, o SIFSM permite transformar registros operacionais em informações organizadas, indicadores e recomendações que auxiliam no planejamento e na utilização responsável dos recursos disponíveis.

A proposta também considera aspectos relacionados à arquitetura computacional, à automação, à privacidade dos dados e à responsabilidade socioambiental, demonstrando como diferentes áreas da tecnologia podem atuar conjuntamente na construção de um ambiente mais eficiente, resiliente e sustentável.

Assim, o SIFSM representa uma proposta de infraestrutura inteligente para apoiar o futuro da Aurora Siger, conectando tecnologia, análise de dados e sustentabilidade em um único sistema integrado.

Objetivos

Objetivo Geral

Desenvolver um protótipo de sistema integrado capaz de monitorar, organizar e analisar informações operacionais da colônia Aurora Siger, utilizando recursos da programação e da ciência de dados para apoiar o gerenciamento de recursos e a tomada de decisões.

Objetivos específicos

* Implementar um banco de dados para armazenar e gerenciar informações dos módulos e sensores da colônia.

* Desenvolver operações de cadastro, consulta, atualização e exclusão de registros (CRUD).

* Aplicar estruturas de dados e regras de lógica booleana para organizar informações e identificar situações de alerta.

* Simular o consumo e a disponibilidade de recursos energéticos por meio de modelos matemáticos.

* Utilizar regressão linear para analisar relações entre variáveis e estimar o consumo energético.

* Propor uma arquitetura multiagente baseada em inteligência artificial para auxiliar na análise e interpretação dos dados.

* Relacionar a aplicação tecnológica à eficiência operacional, à sustentabilidade e ao uso responsável de recursos.

Descrição do sistema e menu

O sistema é executado no terminal em Python, com menu textual. A arquitetura tem três camadas: física (sensores e atuadores simulados), de controle (regras booleanas, filas e alertas) e analítica (simulação, regressão e agentes de IA), todas apoiadas pelo banco de dados.

Opção	Funcionalidade	Requisito atendido
1	Cadastrar módulo, sensor ou leitura	CRUD – Create
2	Listar e consultar registros	CRUD – Read
3	Alterar registro	CRUD – Update
4	Excluir registro	CRUD – Delete
5	Executar regra lógica de decisão	Lógica booleana

Opção	Funcionalidade	Requisito atendido
6	Exibir circuito lógico / tabela verdade	Circuito lógico
7	Executar simulação de consumo e bateria	Simulação + integração
8	Executar modelo linear	Regressão linear
9	Analisar sustentabilidade da colônia	Fluxo integrado + estruturas de dados
0	Sair	–

Fluxo da opção 9 (Análise de sustentabilidade): (1) consulta as leituras no banco; (2) aplica a regra booleana de risco; (3) insere os alertas em uma fila de prioridade; (4) executa a simulação de consumo; (5) estima o consumo futuro por regressão; (6) exibe uma recomendação técnica que deve ser validada por um operador humano.

Divisão do Projeto: O que é cada fase

O sistema foi estruturado em módulos de código, cada um responsável por uma área especifica do projeto. A seguir, estão as principais etapas e os conceitos que cada módulo aborda.

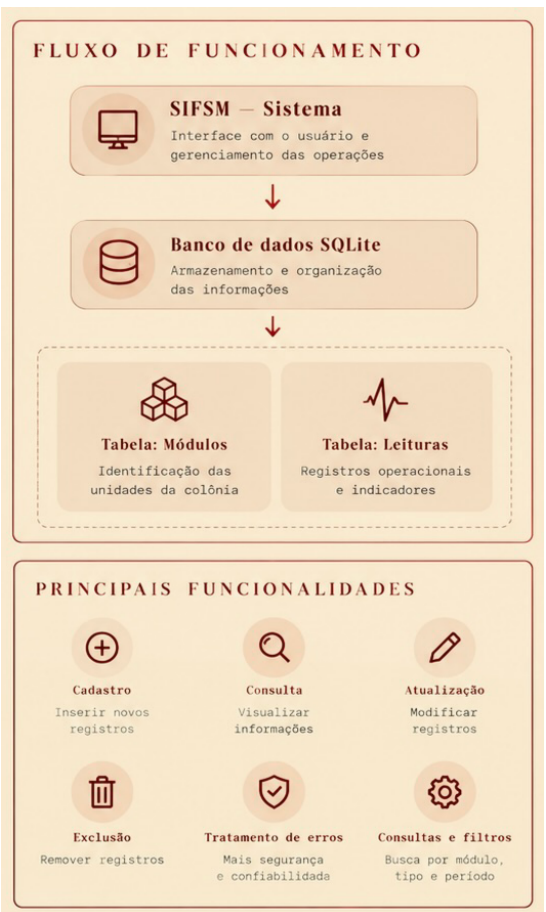


Organização e Gerenciamento dos Dados

O SIFSM utiliza um banco de dados SQLite para armazenar e organizar as informações operacionais da colônia Aurora Siger. Essa estrutura permite centralizar registros e facilitar o acesso aos dados necessários para o monitoramento dos diferentes módulos.

Gerenciamento e Integridade

A utilização de um banco de dados estruturado permite realizar consultas e modificações nos registros, mantendo as informações organizadas para as demais funcionalidades do sistema. O projeto também prevê tratamento de erros durante as operações, contribuindo para a consistência e a confiabilidade dos dados.



Dados Operacionais

As informações são organizadas principalmente em duas categorias:



Gerenciamento dos registros

O SIFSM implementa operações CRUD (Create, Read, Update e Delete) para permitir o gerenciamento das leituras armazenadas no banco de dados SQLite. Essas operações possibilitam inserir, consultar, modificar e excluir informações utilizadas no monitoramento da colônia Aurora Siger.

As quatro operações

01. Create — Cadastro

Permite inserir novas leituras, associando cada registro a um módulo existente. São informados dados como ciclo, temperatura, umidade, quantidade de tripulantes e consumo energético.

02. Read — Consulta

Recupera e apresenta os registros armazenados, organizados por identificador, exibindo as leituras mais recentes primeiro.

03. Update — Atualização

Possibilita modificar o consumo energético de uma leitura já cadastrada, mantendo os demais dados do registro.

04. Delete — Exclusão

Remove uma leitura específica do banco de dados, utilizando seu identificador para localizar o registro correspondente.

Tratamento de erros e segurança

Para garantir maior confiabilidade durante a execução, o sistema utiliza mecanismos de validação e tratamento de exceções. Entradas numéricas são verificadas quanto ao formato e aos limites permitidos, enquanto registros inexistentes e falhas de banco de dados recebem tratamentos específicos.

Além disso, operações de alteração e exclusão verificam a existência do registro antes de realizar modificações. Em caso de falha no banco, o sistema utiliza o comando rollback para cancelar a transação. O projeto também implementa uma pilha de histórico que permite desfazer a última operação de cadastro, atualização ou exclusão.

Organização e processamento das informações

O desenvolvimento do SIFSM utiliza diferentes estruturas de dados da linguagem Python para organizar informações, otimizar consultas e controlar operações internas. Cada estrutura foi escolhida de acordo com sua finalidade, contribuindo para o funcionamento integrado do sistema.

01. Fila (Queue — FIFO)

A fila é utilizada para organizar os alertas gerados durante o monitoramento dos módulos. Seu funcionamento segue o princípio FIFO (First In, First Out), no qual o primeiro elemento inserido é também o primeiro a ser retirado.

No SIFSM, quando uma leitura identifica consumo elevado em um módulo crítico, um alerta é gerado e adicionado à fila. Dessa maneira, os avisos podem ser processados na ordem em que surgem.
Implementação: biblioteca deque, com operações de inserção e remoção de elementos.

02. Pilha (Stack — LIFO)

A pilha armazena o histórico das operações realizadas no banco de dados. Seu funcionamento segue o princípio LIFO (Last In, First Out), no qual o último elemento inserido é o primeiro a ser removido.

Essa estrutura permite implementar a funcionalidade de desfazer a última operação de cadastro, atualização ou exclusão de uma leitura.
Implementação: lista nativa do Python, utilizando operações de inserção e remoção no topo da pilha.

03. Tabela Hash (Dictionary)

A tabela hash é utilizada para organizar os módulos por seus códigos de identificação, permitindo localizar rapidamente suas informações durante a execução do sistema.

No projeto, um dicionário associa cada código às características do respectivo módulo, como nome, estado crítico, situação de atividade e limite de consumo energético.

Implementação: estrutura dict do Python, que oferece buscas eficientes pela chave de identificação.

04. Grafo (Graph)

O grafo representa as relações de dependência entre os módulos da colônia. Essa estrutura permite compreender como uma falha em determinada unidade pode afetar o funcionamento de outras.

No SIFSM, as conexões são representadas por uma lista de adjacência. A partir dela, o algoritmo de busca em largura (BFS) identifica os módulos que podem ser afetados por uma falha em outro componente, como o sistema de energia.

Implementação: dicionário com listas de adjacência e algoritmo BFS (Breadth-First Search).

Estrutura	Aplicação no SIFSM	Justificativa
Lista / vetor	Armazena leituras de sensores por ciclo	Acesso sequencial e base para NumPy/Pandas
Tabela hash (dict)	Busca por código de sensor, módulo ou alerta	Busca em tempo médio O(1)
Fila de prioridade	Alertas e chamados de manutenção ordenados por nível de risco	Atende primeiro o mais crítico (crítico antes de não crítico)
Pilha	Histórico de ações e operação de desfazer	Estrutura LIFO: a última ação é a primeira a ser desfeita
Grafo	Dependências entre módulos (ex.: energia > agricultura > habitação)	Mostra o efeito em cascata de uma falha

Integração das estruturas

A aplicação conjunta dessas quatro estruturas permite que o SIFSM organize alertas, mantenha o histórico de operações, facilite a localização de módulos e analise as dependências da infraestrutura. Assim, conceitos fundamentais da ciência da computação são aplicados a situações práticas de gerenciamento de uma colônia marciana.

Dados, banco de dados e CRUD

Os dados são simulados e representam módulos da colônia (habitação, agricultura, laboratório, comunicação, suporte médico, armazenamento e energia), seus sensores e as leituras periódicas. O banco pode ser Oracle ou equivalente (SQLite, usado como alternativa local); o arquivo dados_aurora_siger.csv serve como base de carga inicial.

Modelo de dados

Tabela	Campos principais	Descrição
modulo	id_modulo (PK), nome, tipo, critico (0/1), status	Módulos da colônia
sensor	id_sensor (PK), id_modulo (FK), tipo, status_valido	Sensores simulados e seus códigos
leitura	id_leitura (PK), id_sensor (FK), data_hora, consumo_kwh, temperatura, umidade, luminosidade	Medições por ciclo
alerta	id_alerta (PK), id_modulo (FK), nivel, descricao, data_hora, resolvido	Alertas gerados pelas regras

```
CREATE TABLE modulo (
  id_modulo INTEGER PRIMARY KEY,
  nome      VARCHAR(50) NOT NULL,
  tipo      VARCHAR(30) NOT NULL,
  critico   NUMBER(1)   DEFAULT 0 CHECK (critico IN (0,1)),
  status    VARCHAR(15) DEFAULT 'ATIVO'
);
CREATE TABLE sensor (
  id_sensor INTEGER PRIMARY KEY,
  id_modulo INTEGER REFERENCES modulo(id_modulo),
  tipo      VARCHAR(30) NOT NULL,
  status_valido NUMBER(1) DEFAULT 1
);
CREATE TABLE leitura (
  id_leitura  INTEGER PRIMARY KEY,
  id_sensor   INTEGER REFERENCES sensor(id_sensor),
  data_hora   TIMESTAMP,
  consumo_kwh NUMBER(8,2) CHECK (consumo_kwh >= 0),
  temperatura NUMBER(5,2),
  umidade     NUMBER(5,2)
);
```

Operações CRUD e tratamento de erros

Cada operação é uma função Python com comandos SQL parametrizados (evitando injeção de SQL). O tratamento de erros usa **try/except/else/finally:** ValueError para entradas inválidas (ex.: consumo negativo ou texto no lugar de número), erros de banco para chaves duplicadas ou inexistentes, e finally para fechar a conexão.

```
def cadastrar_leitura(conn, id_sensor, consumo, temp, umid):
    try:
        consumo = float(consumo)
        if consumo < 0:
            raise ValueError("Consumo não pode ser negativo.")
        cur = conn.cursor()
        cur.execute("INSERT INTO leitura (id_sensor, consumo_kwh, temperatura, umidade)"
                    " VALUES (?, ?, ?, ?)", (id_sensor, consumo, temp, umid))
    except ValueError as e:
        print(f"Entrada inválida: {e}")
    except Exception as e:
        print(f"Falha na operação: {e}")
        conn.rollback()
    else:
        conn.commit()
        print("Leitura cadastrada com sucesso.")
    finally:
        cur.close() if 'cur' in locals() else None
```

Regra booleana e circuito lógico

Definição das variáveis: A = consumo alto (acima do limite do módulo); B = módulo crítico; C = sensor em estado inválido.

Expressão booleana: ALERTA = (A · B) + C, ou seja, aciona-se o alerta se o consumo estiver alto e o módulo for crítico, ou se o sensor estiver inválido (operação bloqueada por segurança).

```
alerta = (consumo_alto and modulo_critico) or sensor_invalido
```

A (consumo alto)	B (módulo crítico)	C (sensor inválido)	ALERTA
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Circuito lógico: uma porta AND (A, B) alimenta uma porta OR junto com C; a saída aciona um LED de alerta. No Tinkercad, A, B e C são chaves (ou botões) e a lógica pode ser montada com CIs 7408 (AND) e 7432 (OR) ou com Arduino (pinos digitais de entrada e saída, resistor de 220 ohms no LED).

O circuito é entregue em **circuito_logico.pdf/png ou link_tinkercad.txt**. O mesmo resultado é obtido em software (Python) e em hardware (lógica digital), demonstrando a equivalência entre as duas representações.

Simulação Computacional e Integração Matemática

A simulação representa o consumo energético dos módulos e o estado de carga (SoC) da bateria ao longo de um ciclo marciano, comparando cenários (ex.: operação normal x modo econômico).

Variáveis: tempo t (h), geração solar Pger(t) (kW), consumo Pcons(t) (kW), capacidade C (kWh) e SoC inicial.

Modelo matemático: SoC(t) = SoC0 + (1/C) · integral de t0 a t de [Pger(s) − Pcons(s)] ds, calculado por integração numérica (regra do trapézio). A energia total consumida é a área sob a curva de potência, e a média da função fornece a potência média do ciclo.

```
import numpy as np
t = np.linspace(0, 24, 241)                        # horas
p_ger = np.maximum(0, 5*np.sin(np.pi*(t-6)/12))   # geração solar (kW)
p_cons = 2.2 + 0.4*np.sin(2*np.pi*t/24)           # consumo (kW)
saldo = p_ger - p_cons
soc = 0.6 + np.cumsum((saldo[:-1]+saldo[1:])/2*np.diff(t))/50   # C = 50 kWh
soc = np.clip(soc, 0, 1)
energia_kwh = np.trapz(p_cons, t)                  # área sob a curva
```

Limitações: o modelo ignora perdas de carga/descarga, degradação da bateria, tempestades de poeira e variações sazonais; os dados são sintéticos.

Modelo linear aplicado à decisão

Objetivo: estimar o consumo energético (kWh) de um módulo a partir de variáveis operacionais.

Modelo múltiplo proposto:

consumo_kwh = b0 + b1 * temperatura_externa + b2 * ocupacao_modulo + erro

- **Análise exploratória:** estatísticas descritivas, histogramas, gráficos de dispersão e matriz de correlação.
- **Interpretação dos coeficientes:** b1 indica a variação média do consumo para cada grau adicional de temperatura (mantida a ocupação constante); b2 indica o efeito de cada tripulante adicional; b0 é o consumo-base.
- **Avaliação:** R^2, RMSE/MAE e análise de resíduos em uma parte dos dados separada para teste.

Cuidados: multicolinearidade entre variáveis, outliers, extrapolação fora da faixa observada e correlação não implica causalidade.

Agentes, fluxos e guardrails de IA

A IA é proposta como parte de uma arquitetura maior, de forma simulada/conceitual, sempre conectada a dados, ferramentas e regras.

Componente	Papel
Agente triador	Interpreta a solicitação da tripulação e encaminha ao agente adequado (handoff)
Agente de energia	Consulta consumo e SoC; usa a simulação e a regressão como ferramentas
Agente ambiental	Analisa temperatura, umidade e luminosidade dos módulos
Agente de manutenção	Abre chamados na fila de prioridade e consulta histórico
Agente de dados (RAG simulado)	Responde com base em registros do próprio banco e no relatório técnico

- **Ferramentas determinísticas:** funções Python que consultam o banco e calculam indicadores, evitando que o modelo "invente" números.
- **Guardrails:** recusar ações fora do escopo; impedir comandos diretos em sistemas críticos (suporte de vida, por exemplo) sem aprovação humana; validar entradas e saídas; não expor dados pessoais da tripulação.
- **Sessões e tracing:** histórico de interação e registro das chamadas de ferramentas para auditoria.
- **Responsabilidade humana:** a IA apenas sugere; a decisão final é do operador. Os limites e a possibilidade de erro são informados ao usuário.

Arquitetura computacional

- **CPU:** executa o menu, o CRUD e as regras booleanas (tarefas de controle, pouco paralelas).
- **GPU:** indicada para operações vetoriais com muitos dados (simulações em larga escala e treino de modelos); no protótipo, NumPy já explora vetorização na CPU.
- **NPU/TPU:** aceleradores eficientes em energia para inferência de IA local, relevantes onde a energia é escassa.
- **Computação heterogênea:** cada tarefa na unidade mais adequada (controle na CPU, inferência no acelerador).
- **Hierarquia de memória e movimentação de dados:** o gargalo costuma estar no transporte entre memória e processador; manter dados frequentes em cache/memória local e processar na borda (perto dos sensores) reduz tráfego e consumo.
- **Sustentabilidade:** menor processamento significa menor consumo energético, algo crítico em Marte; é preferível executar localmente e evitar cálculos redundantes.



Sustentabilidade, cidadania digital e impacto socioambiental

- **Uso responsável dos recursos:** a integração entre sensores, regras e simulação permite detectar desperdícios de energia e agir antes de uma falha.
- **Simulação de cenários:** comparar estratégias no software evita testar em campo e reduz desperdício de materiais e energia.
- **Autonomia e resiliência:** armazenamento em bateria, redes inteligentes e automação mantêm módulos críticos funcionando em contingências.
- **Cidadania digital e privacidade:** dados da tripulação (presença, saúde, rotina) devem ser coletados só quando necessário, com transparência e controle de acesso.
- **Economia dos dados:** uso indevido de informações pessoais pode gerar vigilância e desigualdade; por isso o sistema adota minimização de dados e anonimização.
- **Impacto de IA e data centers:** processamento intensivo consome energia e água; o projeto prioriza modelos leves, processamento local e cálculos determinísticos.
- **Inclusão e responsabilidade social:** interfaces claras e acessíveis e decisões explicáveis para toda a tripulação.

Limitações e melhorias futuras

- Dados simulados, sem validação com medições reais.
- Modelo linear simplificado: não captura não linearidades nem eventos extremos.
- Agentes de IA propostos de forma conceitual/simulada, sem integração real de LLM.
- Melhorias: dashboard gráfico, sensores físicos reais (Arduino), modelos preditivos mais robustos, autenticação de usuários e auditoria completa.

Estrutura da entrega e execução

Arquivo	Conteúdo
codigo_fonte.py / notebook_projeto.ipynb	Sistema principal com menu, CRUD, regras e análises
script_banco.sql	Criação das tabelas
dados_aurora_siger.csv	Base de dados simulada
circuito_logico.pdf / link_tinkercad.txt	Circuito lógico e simulação
simulacao_computacional.py	Simulação de consumo e bateria
relatorio_tecnico.pdf	Este relatório

Arquivo	Conteúdo
README.md	Objetivo, arquivos, dependências e execução
link_video.txt	Link do vídeo (YouTube, não listado)
graficos_ou_imagens/	Gráficos e evidências de execução (opcional)

Dependências: Python 3.10+, numpy, pandas, matplotlib, scikit-learn (instalação: pip install numpy pandas matplotlib scikit-learn). Execução: python codigo_fonte.py.

Conclusão

O desenvolvimento do Sistema Integrado de Futuro Sustentável em Marte (SIFSM) demonstrou como a integração entre programação, estruturas de dados, lógica computacional, análise matemática e inteligência artificial pode contribuir para o gerenciamento de uma infraestrutura habitacional em um ambiente extraterrestre.

Ao longo do projeto, foram implementados recursos para organizar informações operacionais, registrar e consultar leituras, controlar alterações, identificar situações de risco e analisar o comportamento energético da colônia. As simulações matemáticas, a regressão linear e a arquitetura multiagente ampliaram as possibilidades de análise e apoio à tomada de decisões.

Embora o protótipo utilize dados simulados e apresente limitações que exigem aprimoramentos futuros, sua estrutura estabelece uma base para evoluções como a integração com sensores reais, modelos preditivos mais avançados e sistemas inteligentes de monitoramento.

Assim, o Aurora Siger evidencia o potencial da tecnologia como ferramenta de planejamento, eficiência e sustentabilidade, demonstrando que a construção de uma colônia em Marte depende não apenas de recursos físicos, mas também de sistemas capazes de transformar dados em informações úteis para decisões mais seguras e conscientes.